1. Why Lists Matter in Data Engineering

A real pipeline usually works with multiple items.

For example:

```
received_files = [
    "customers.csv",
    "orders.csv",
    "products.csv"
]
```

Instead of creating a separate variable for every file, a list lets us manage all related values together.

Common Data Engineering uses:

- Received files
- Required columns
- Processed files
- Failed files
- Pipeline statuses
- Valid records
- Rejected records

---

2. What Is a List?

A list is an **ordered and mutable collection** that can store multiple values.

```
received_files = [
    "customers.csv",
    "orders.csv",
    "products.csv"
]
```

Check the type:

```
print(type(received_files))
```

Output:

<class 'list'>

Lists are created using:

[]

---

3. Accessing List Elements

List indexing starts from `0`.

```
received_files = [
    "customers.csv",
    "orders.csv",
    "products.csv"
]
```

Indexes:

0 → customers.csv
1 → orders.csv
2 → products.csv

Access the first file:

```
print(received_files[0])
```

Output:

customers.csv

---

4. Negative Indexing

Negative indexes access values from the end.

```
print(received_files[-1])
```

Output:

products.csv

Useful when you need the latest or last item.

---

5. Finding List Length

Use `len()`:

file_count = len(received_files)

print(f"Files Received: {file_count}")

Output:

Files Received: 3

In a pipeline, this can help track how many files or records were received.

---

6. List Slicing

Slicing extracts part of a list.

```
files = [
    "customers.csv",
    "orders.csv",
    "products.csv",
    "payments.csv"
]

print(files[:2])
```

Output:

```
[
   "customers.csv",
   "orders.csv"
]
```

Remember:

- Start index is included.
- End index is excluded.

---

7. Lists Are Mutable

Unlike strings, lists can be modified after creation.

```
files = [
   "customers.csv",
   "order.csv"
]

files[1] = "orders.csv"
```

Now:

```
print(files)
```

Output:

```
[
   "customers.csv",
   "orders.csv"
]
```

This is called **mutability**.

---

8. Adding Items with `append()`

`append()` adds one item to the end of a list.

```
processed_files = []

processed_files.append("customers.csv")
processed_files.append("orders.csv")
```

Result:

```
[
   "customers.csv",
   "orders.csv"
]
```

This is a very common pipeline pattern.

---

9. `append()` vs `extend()`

Suppose:

```
files = ["customers.csv"]

new_files = [
   "orders.csv",
   "products.csv"
]
```

Using:

```
files.append(new_files)
```

creates:

```
[
   "customers.csv",
   [
      "orders.csv",
      "products.csv"
```

```
    ]
]
```

The complete second list is added as one item.

If we want each file separately:

```
files.extend(new_files)
```

Result:

```
[
    "customers.csv",
    "orders.csv",
    "products.csv"
]
```

Remember

append() → Add one item
extend() → Add multiple items

---

10. Adding at a Specific Position

Use `insert()`.

```
files = [
    "customers.csv",
    "products.csv"
]

files.insert(1, "orders.csv")
```

Result:

```
[
    "customers.csv",
    "orders.csv",
```

```
    "products.csv"
]
```

---

11. Removing Items

`remove()`

Removes using the value.

```
files = [
    "customers.csv",
    "test_file.csv",
    "orders.csv"
]

files.remove("test_file.csv")
```

`pop()`

Removes using an index and returns the removed value.

```
files = [
    "customers.csv",
    "orders.csv",
    "products.csv"
]

removed_file = files.pop(1)
```

Now:

```
removed_file = orders.csv
```

and the list becomes:

```
[
```

```
    "customers.csv",
    "products.csv"
]
```

Remember

remove() → Remove by value
pop() → Remove by index

---

12. Checking Whether an Item Exists

Use `in` and `not in`.

```
required_columns = [
    "customer_id",
    "customer_name",
    "email"
]
```

Check:

```
if "customer_id" in required_columns:
    print("Customer ID column available")
```

Another example:

```
if "phone_number" not in required_columns:
    print("Phone number is not required")
```

This is useful for schema validation.

---

13. Counting Values

Use `count()`.

```
pipeline_statuses = [
    "SUCCESS",
    "FAILED",
    "SUCCESS",
    "SUCCESS"
]

success_count = pipeline_statuses.count("SUCCESS")
failure_count = pipeline_statuses.count("FAILED")
```

Results:

```
Success: 3
Failed: 1
```

Useful for pipeline summary reporting.

---

14. Finding an Item Position

Use `index()`.

```
required_columns = [
    "customer_id",
    "customer_name",
    "email"
]

position = required_columns.index("email")
```

Output:

```
2
```

Be careful: `index()` raises an error if the value does not exist.

Safer:

```
if "email" in required_columns:
```

```
print(required_columns.index("email"))
```

---

15. Sorting Lists

Suppose:

```
file_sizes = [
    250,
    100,
    500,
    150
]
```

Ascending:

```
file_sizes.sort()
```

Result:

```
[100, 150, 250, 500]
```

Descending:

```
file_sizes.sort(reverse=True)
```

Result:

```
[500, 250, 150, 100]
```

---

16. `sort()` vs `sorted()`

`sort()` changes the original list.

```
file_sizes.sort()
```

`sorted()` creates a new sorted result.

```
file_sizes = [250, 100, 500]

sorted_sizes = sorted(file_sizes)
```

Now:

```
Original: [250, 100, 500]
Sorted:   [100, 250, 500]
```

Interview Point

sort() → Modifies original list
sorted() → Returns a new sorted result

---

17. Real Scenario: Supported and Unsupported Files

Suppose a pipeline receives:

```
received_files = [
    "customers.csv",
    "orders.json",
    "products.xml",
    "payments.csv",
    "readme.txt"
]
```

Supported formats:

```
supported_extensions = (
    ".csv",
    ".json",
    ".parquet"
)
```

Create:

```
supported_files = []
unsupported_files = []
```

For one file:

```
file_name = received_files[0]

if file_name.endswith(supported_extensions):
    supported_files.append(file_name)
else:
    unsupported_files.append(file_name)
```

The same logic can separate valid and invalid files.

For many files, repeating this manually is inefficient.

That is why lists naturally lead to **loops**.

---

18. Valid and Rejected Records

Lists can also separate records based on validation.

```
valid_records = []
rejected_records = []

customer_id = 101
order_amount = 2500

if customer_id is not None and order_amount >= 0:
    valid_records.append(customer_id)
else:
    rejected_records.append(customer_id)
```

This is the basic idea behind separating clean and rejected data.

---

19. Lists and Column Cleaning

Suppose:

```
raw_columns = [
    " Customer ID ",
    "Customer Full Name",
    "ORDER-AMOUNT"
]
```

One value can be cleaned using:

```
column = raw_columns[0]

clean_column = (
    column
    .strip()
    .lower()
    .replace("-", " ")
)

clean_column = "_".join(
    clean_column.split()
)
```

Result:

```
customer_id
```

Doing this for every list item manually would be repetitive.

Again, this leads naturally to loops.

---

20. Common Mistakes

Mistake 1: Assuming Index Starts from 1

Wrong:

First item → index 1

Correct:

First item → index 0

Mistake 2: Invalid Index

```
files = [
    "customers.csv",
    "orders.csv"
]

print(files[5])
```

This causes:

IndexError

Mistake 3: Confusing `append()` and `extend()`

```
files.append(
    ["orders.csv", "products.csv"]
)
```

adds a nested list.

Use:

```
files.extend(
    ["orders.csv", "products.csv"]
)
```

to add both items separately.

Mistake 4: Assuming `remove()` Uses Index

Wrong:

files.remove(1)

unless `1` is actually stored in the list.

Use:

files.pop(1)

to remove using an index.

Mistake 5: Forgetting Lists Are Mutable

This changes the original list:

files[0] = "new_file.csv"

Unlike strings, lists can be modified.

---

21. Quick Revision

- Lists store multiple values in one variable.
- Lists are ordered.
- Lists are mutable.
- Indexing starts from `0`.
- Negative indexing starts from the end.
- `len()` gives the number of items.
- `append()` adds one item.
- `extend()` adds multiple items.
- `insert()` adds at a specific position.
- `remove()` removes using value.
- `pop()` removes using index.

- `in` and `not in` help validate membership.
- `count()` counts occurrences.
- `index()` finds a position.
- `sort()` changes the original list.
- `sorted()` returns a sorted result.

---

22. Student Practice

Create:

```
received_files = [
    "customers.csv",
    "orders.json",
    "products.xml",
    "payments.parquet"
]
```

Perform these tasks:

1. Print the total number of files.
2. Print the first file.
3. Print the last file.
4. Add `"transactions.csv"`.
5. Remove `"products.xml"`.
6. Check whether `"orders.json"` exists.
7. Print the final list.

Expected final list:

```
[
    "customers.csv",
    "orders.json",
    "payments.parquet",
    "transactions.csv"
]
```

---

23. Additional Practice Scenario

Create two lists:

supported_files = []
unsupported_files = []

Use these filenames:

file_1 = "customers.csv"
file_2 = "products.xml"

Validate their extensions and place each file into the correct list.

Supported formats:

```
(
    ".csv",
    ".json",
    ".parquet"
)
```

---

24. Interview-Relevant Questions

1. What is a Python list?

A list is an ordered and mutable collection used to store multiple values.

2. Are lists mutable?

Yes. Items can be added, removed, or updated after the list is created.

3. What is the difference between `append()` and `extend()`?

`append()` adds one object.

`extend()` adds each item from another iterable separately.

4. What is the difference between `remove()` and `pop()`?

`remove()` removes using the value.

`pop()` removes using the index and returns the removed item.

5. What is the difference between `sort()` and `sorted()`?

`sort()` modifies the original list.

`sorted()` returns a new sorted result.

6. How are lists used in Data Engineering?

Lists can store:

- Files
- Column names
- Table names
- Validation results
- Processed items
- Failed items
- Valid records
- Rejected records

7. What happens if `index()` cannot find the requested value?

Python raises an error.

A membership check using `in` can be performed first.

8. Why are lists important before learning loops?

Lists store collections of items, while loops allow us to process each item automatically.

---

25. Mini Interview Challenge

Predict:

files = ["customers.csv"]

```
new_files = [
    "orders.csv",
    "products.csv"
]

files.append(new_files)

print(len(files))
```

Answer:

2

Why?

Because the entire `new_files` list was added as **one item**.

Now:

```
files = ["customers.csv"]

files.extend([
    "orders.csv",
    "products.csv"
])

print(len(files))
```

Output:

3

Key Takeaway

Multiple Files / Columns / Records
↓
List
↓
Store and Manage Together
↓

Loops Process Them Automatically

A list allows a Data Engineer to manage collections of files, columns, records, and processing results without creating separate variables for every item.